

Tohjulet selvbalancerende robot.

Selvstændigt projekt i fysik A af
Hamza Alomari 3.B

Vejleder:
Kasper Weirum Risgaard (KWR)

Frederikshavn Teknisk Gymnasium
HTX

16/03/2026 – 27/04/2026

Tegn: 24012 $\approx$ 10 normalside

Resume

Dette projekt undersøger hvordan en tohjulet selvbalancerende robot kan holdes i stabil balance ved hjælp af et PID-baseret reguleringssystem. Systemet modelleres som et inverteret pendul, og bevægelsesligningen udledes ved hjælp af Lagrangian mekanik. Robottens hældningsvinkel estimeres med en MPU-9250/6050 IMU-sensor, hvor et komplementærfilter kombinerer gyroskop- og accelerometerdata. Forsøgene viste at komplementærfilteret estimerer vinklen med små afvigelser, og at PID-regulatoren holder robotten stabil ved $\pm 2^\circ$ omkring setpunktet. Den maksimale genopretningsvinkel blev bestemt til intervallet $15^\circ < \theta_{max} < 20^\circ$, hvorefter robotten vælter i overensstemmelse med den teoretiske model.

Indholdsfortegnelse

Indledning	4
1.1 Problemstilling	4
1.2 Formål med forsøg	4
Teoretisk fundament for robottens opbygning og funktionalitet	5
2.1 Lagrangian mekanik og udledning af bevægelsesligning for det inverterede pendul	5
2.1.1 Basis for Lagrangian mekanik	5
2.1.2 Udledning af bevægelsesligning for det inverterede pendul	6
2.2 MPU-6050/9250 og komplementærfilter	10
2.3 PID-regulering	14
2.3.1 Opbygning af PID-regulator	14
2.3.2 Tuning af PID-regulatoren	16
Materialer	18
4.1 Robottens skematiske diagram	19
4.2 Delforsøg #1	19
4.3 Delforsøg #2 & #3	20
Fremgangsmåde	22
5.1 Delforsøg #1	22
5.2 Delforsøg #2	22
5.3 Delforsøg #3	23
Resultater	24
6.1 Delforsøg 1	24
6.2 Delforsøg 2	25
6.3 Delforsøg 3	25
Diskussion	27
Konklusion	29
Referencer	30
Bilag	31
Udledning af bevægelsesligning for den stive krop med et inertimoment I	32
IMU kode til delforsøg 1	34
Balanceringskode til delforsøg 2 & 3	36

Nomenklatur

Forkortelser

IMU	Inertial Measurement Unit
MEMS	Micro-Electro-Mechanical System
PID	Proportional-Integral-Derivative
ADC	Analog-to-Digital Converter
I2C	Inter-Integrated Circuit
MCU	Microcontroller Unit
DMP	Digital Motion Processor
PWM	Pulse Width Modulation

Notation

$\frac{d}{dt}$	Tidsafledt (differential operator)
$\frac{d\theta}{dt} = \dot{\theta}$	Første tidsafledte (vinkelhastighed)
$\frac{d^2\theta}{dt^2} = \ddot{\theta}$	Anden tidsafledte (vinkelacceleration)
$\frac{\partial}{\partial q_i}$	Partiel afledt mht. koordinat
$\frac{\partial L}{\partial q_i}$	Ændring i Lagrangian mht. position
$\frac{\partial L}{\partial \dot{q}_i}$	Ændring i Lagrangian mht. hastighed

KAPITEL 1

Indledning

En tohjulet selvbalancerende robot er et eksempel på et ustabilt mekanisk system. Systemet kan beskrives som et omvendt pendul. Små hældninger vil derfor få robotten til at vælte, hvis der ikke sker en aktiv regulering. Robotten måler sin hældningsvinkel med en IMU og justerer motorernes hastighed for at genoprette balancen.

1.1 Problemstilling

Hvordan kan et reguleringssystem baseret på PID-kontrol holde en tohjulet robot i stabil balance ved hjælp af en Lagrangian mekanisk model af et inverteret pendul?

1.2 Formål med forsøg

Forsøg 1: Karakterisering af MPU-9250/6050 sensoren og implementering af et komplementærfilter til bestemmelse af robottens hældningsvinkel.

Forsøg 2: Opstilling og justering af PID-parametre.

Forsøg 3: At bestemme robottens maksimale genopretningsvinkel θ_{max} , dvs. den største hældning hvorfra PID-regulatoren stadig er i stand til at stabilisere systemet

Teoretisk fundament for robottens opbygning og funktionalitet

2.1 Lagrangian mekanik og udledning af bevægelsesligning for det inverterede pendul

For at forsimple de teoretiske udregninger beskrives systemets mekanik ved hjælp af Lagrangian mekanik i stedet for traditionel newtonsk mekanik.

2.1.1 Basis for Lagrangian mekanik

Lagrangian mekanik er en alternativ beskrivelse af klassisk mekanik, udviklet af den italiensk-franske matematiker Joseph-Louis Lagrange i slutningen af 1700-tallet. Metoden adskiller sig fra den newtonske metode ved, at man i stedet for at opstille kraftligninger, tager det udgangspunkt i systemets energier. Dette gør det muligt at analysere komplekse mekaniske systemer på en mere overskuelig måde, særligt når systemet er afhængigt af flere ting, såsom et dobbelt pendul.

Kernen i metoden er Lagrange funktionen, kaldet Lagrangian, defineret som:

$$L = T - V \tag{2.1.1}$$

hvor T er systemets samlede kinetiske energi og V er den potentielle energi. I stedet for kartesiske koordinater (x, y, z) beskrives systemet ved hjælp af generaliserede koordinater q_i , som er et minimalt sæt af variable, der beskriver systemets (Morin, 2007).

For et simpelt pendul vil det eksempelvis blot være vinklen θ .

Systemets bevægelsesligninger udledes direkte fra Lagrangian ved hjælp af Euler-Lagrange ligningen:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \frac{\partial L}{\partial q_i} = 0 \quad (2.1.2)$$

Denne ligning giver en differentiallyigning for hver generaliseret koordinat q_i .

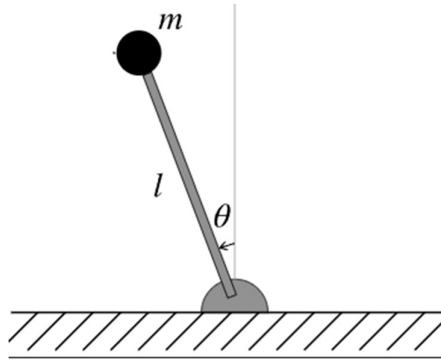
Hvor:

- L : Lagrangianen
- $\dot{q}_i = \frac{dq_i}{dt}$: Tidsafledte af koordinaterne, altså hastigheder.
- $\frac{\partial L}{\partial \dot{q}_i}$: den partielle afledte af Lagrangian med hensyn til hastigheden, og svarer til den generaliserede impuls (momentum): $p_i = \frac{\partial L}{\partial \dot{q}_i}$
- $\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right)$: Tidsafledte af impulsen. Det svarer til ændringen af momentum over tid, altså en slags generaliseret kraft.
- $\frac{\partial L}{\partial q_i}$: Dette er den partielle afledte af Lagrangian med hensyn til selve koordinaten, og viser hvordan energien ændrer sig, når man ændrer positionen

Ligningen udtrykker i bund og grund en variant af Newtons 2. lov, men formuleret i energitermer i stedet for kræfter. Den afgørende fordel er, at man undgår at skulle lave komponenter af alle virkende kræfter, hvilket især er tidskrævende i systemer med bindinger eller rotationsbevægelse (Morin, 2007) .

2.1.2 Udledning af bevægelsesligning for det inverterede pendul

Det inverterede pendul er den fysiske model for den selvbalancerende robot. Man betragter et idealiseret system bestående af en punktmasse m placeret i enden af en masseløs stang med længde l . Vinklen θ måles fra den lodrette akse, så $\theta = 0$ svarer til at pendulet står lodret opad, som er den ustabile ligevægtsposition, som vist i figur 1.



Figur 1 Et inverteret pendul med, en punktmasse m , længde l fra omdrejningspunkt og vinkel θ mellem l og omdrejningspunktet

For udleede systemets bevægelsesligning, startes der med at udregne Lagrangian:

Massens afstand fra ophængs punktet er l , og dens hastighed ved rotation med vinkelhastighed $\dot{\theta}$ er:

$$v = l\dot{\theta} \quad (2.1.3)$$

Den kinetiske energi for en punktmasse bliver derfor:

$$T = \frac{1}{2}mv^2 = \frac{1}{2}ml^2\dot{\theta}^2 \quad (2.1.4)$$

Nulpunktet for den potentielle energi placeres ved ophængs punktet.

Massens højde over dette punkt er $h = l \cdot \cos(\theta)$, og den potentielle energi er:

$$V = mgl\cos(\theta) \quad (2.1.5)$$

Bemærk, at når $\theta = 0$ (lodret oprejst) er $V = mgl$, dvs. størst mulig potentiel energi. Dette er en ustabil ligevægt: enhver lille afvigelse sænker den potentielle energi og vil accelerere systemet yderligere væk fra ligevægt.

Den samlede Lagrangianen for systemet er:

$$L = T - V = \frac{1}{2}ml^2\dot{\theta}^2 - mgl\cos(\theta) \quad (2.1.6)$$

Lagrangian indsættes nu i ligning 2.6.1 og der løses:

$$\begin{aligned} \frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \frac{\partial L}{\partial q_i} &= 0 \\ \frac{d}{dt} \left(\frac{\partial \frac{1}{2}ml^2\dot{\theta}^2 - mgl\cos(\theta)}{\partial \dot{\theta}} \right) - \frac{\partial \frac{1}{2}ml^2\dot{\theta}^2 - mgl\cos(\theta)}{\partial \theta} &= 0 \end{aligned} \quad (2.1.7)$$

Der beregnes de nødvendige partielle afledte:

$$\frac{\partial L}{\partial \dot{\theta}} = \left(\frac{\partial \frac{1}{2} ml^2 \dot{\theta}^2 - mgl \cos(\theta)}{\partial \dot{\theta}} \right) = ml^2 \dot{\theta} \quad (2.18)$$

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}} \right) = \frac{d}{dt} \left(\frac{\partial \frac{1}{2} ml^2 \dot{\theta}^2 - mgl \cos(\theta)}{\partial \dot{\theta}} \right) = ml^2 \ddot{\theta} \quad (2.19)$$

$$\frac{\partial L}{\partial \theta} = \frac{\partial \frac{1}{2} ml^2 \dot{\theta}^2 - mgl \cos(\theta)}{\partial \theta} = mgl \sin(\theta) \quad (2.1.10)$$

Indsættes i Euler-Lagrange ligningen:

$$ml^2 \ddot{\theta} - mgl \sin(\theta) = 0 \quad (2.1.11)$$

hvilket reduceres til bevægelsesligningen for det inverterede pendul:

$$\boxed{\ddot{\theta} = \frac{g}{l} \sin(\theta)} \quad (2.1.12)$$

Bevægelsesligningen er ikke-lineær på grund af $\sin(\theta)$ -leddet, og den har ingen analytisk løsning. For at undersøge systemets stabilitet tæt på ligevægtspositionen $\theta = 0$ lineariseres ligningen ved hjælp af småvinkelapproksimationen, som er en simpel matematisk forenkling af den trigonometriske funktion $\sin(\theta)$:

$$\sin(\theta) \approx \theta \text{ for } \theta \approx 0$$

Dette er en god approksimation for $|\theta| \lesssim 15^\circ$ (Johnson-Steigelman, n.d.).

Bevægelsesligningen bliver da:

$$\ddot{\theta} = \frac{g}{l} \theta \quad (2.1.13)$$

Dette er en lineær, homogen 2. ordens differentialligning med konstante koefficienter. Den generelle løsning er (fundet vha. CAS-værktøjet maple):

$$\theta(t) = C_1 e^{\sqrt{\frac{g}{l}} t} + C_2 e^{-\sqrt{\frac{g}{l}} t} \quad (2.1.14)$$

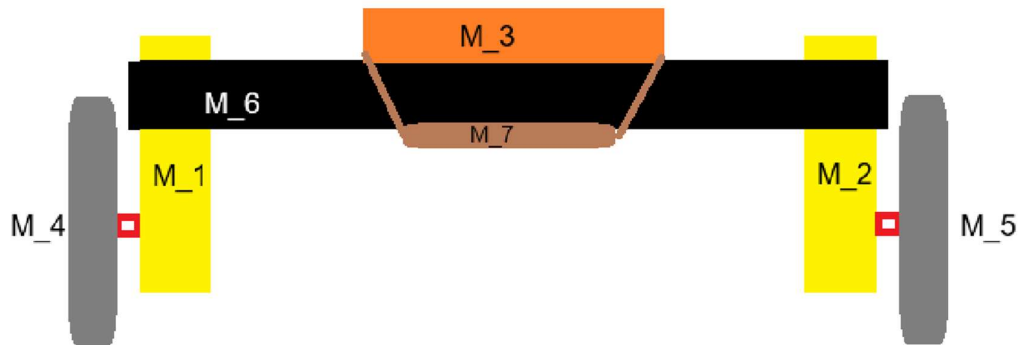
Den første term $C_1 e^{\sqrt{g/l} t}$ vokser eksponentielt over tid, medmindre $C_1 = 0$ præcist, hvilket kræver perfekt startbetingelse. I praksis vil enhver lille forstyrrelse medføre, at $C_1 \neq 0$, og systemet vil vælte. Det inverterede pendul er altså ustabil ved $\theta = 0$.

Vækstraten for differentialligningen er:

$$\lambda = \sqrt{\frac{g}{l}}$$

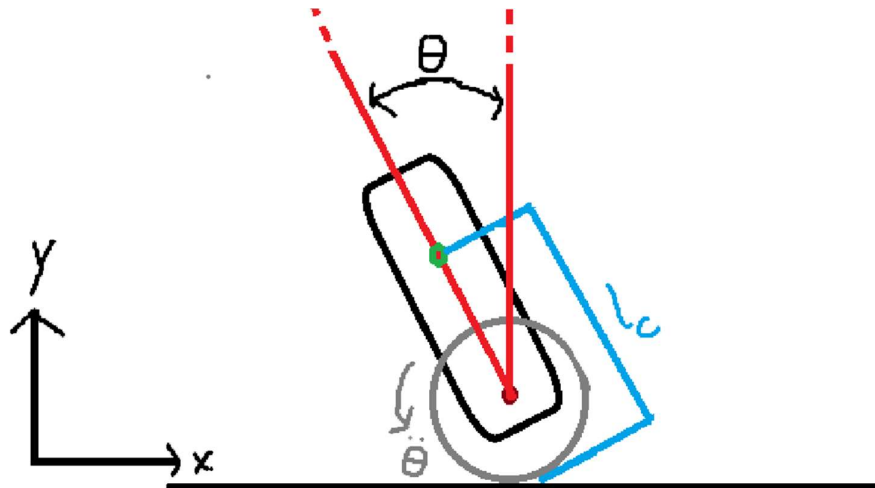
For en robot med $l = 0.20$ m giver dette 7.0 s^{-1} dvs. en fordobling af hældningsvinklen for hver ≈ 0.14 s, hvis der ikke reguleres. Dette illustrerer, hvor hurtigt regulatoren skal reagere.

I praksis er robottens krop ikke en punktmasse, men en krop med en bestemt massefordeling, som vist i figur 2.



Figur 2 Et eksempel på masse fordeling af en selvbalancerende robot

I så fald skal man anvende inertimomentet I om rotationsaksen (hjulenes kontaktpunkt med gulvet) i stedet for ml^2 som er inertimomentet af punktmassen. Tyngdepunktet befinder sig i afstanden l_c fra rotationspunktet som vist i figur 3.



Figur 3 Fritlegemediagram af robotten med indtegnet vinkelacceleration $\ddot{\theta}$, tyngdepunkt (markeret med grønt) og længden l_c

Bevægelsesligningen for den stive krop bliver (fuld udledning findes i Bilag A):

$$I\ddot{\theta} = mgl_c \sin(\theta) \quad (2.1.15)$$

eller for små vinkler:

$$\ddot{\theta} = \frac{mgl_c}{I} \theta \quad (2.1.16)$$

Den tilsvarende vækstrate er:

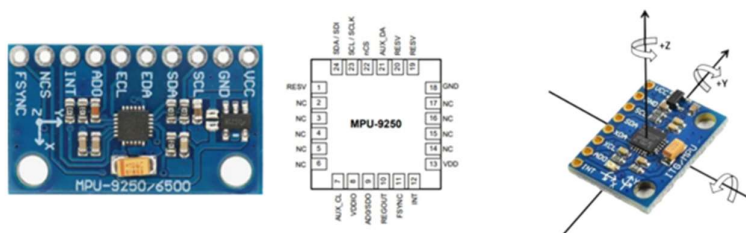
$$\lambda = \sqrt{\frac{mgl_c}{I}}$$

Hvis tyngdepunktet l_c placeres lavere (tættere på hjulene), bliver λ mindre, og systemet vælter langsommere. Det giver regulatoren mere tid til at reagere. Omvendt giver et højt tyngdepunkt en større λ , hvilket stiller skarpere krav til reguleringssystemets reaktionstid. Dette er en vigtig del ved design af den selvbalancerende robot: man ønsker typisk at placere de tungeste komponenter (batteri, motorer) så lavt som muligt.

Denne teoretiske model danner grundlaget for forståelse af, hvilke krav der stilles til reguleringssystemet, herunder den PID-regulator, der beskrives i afsnit 2.3.

2.2 MPU-6050/9250 og komplementærfilter

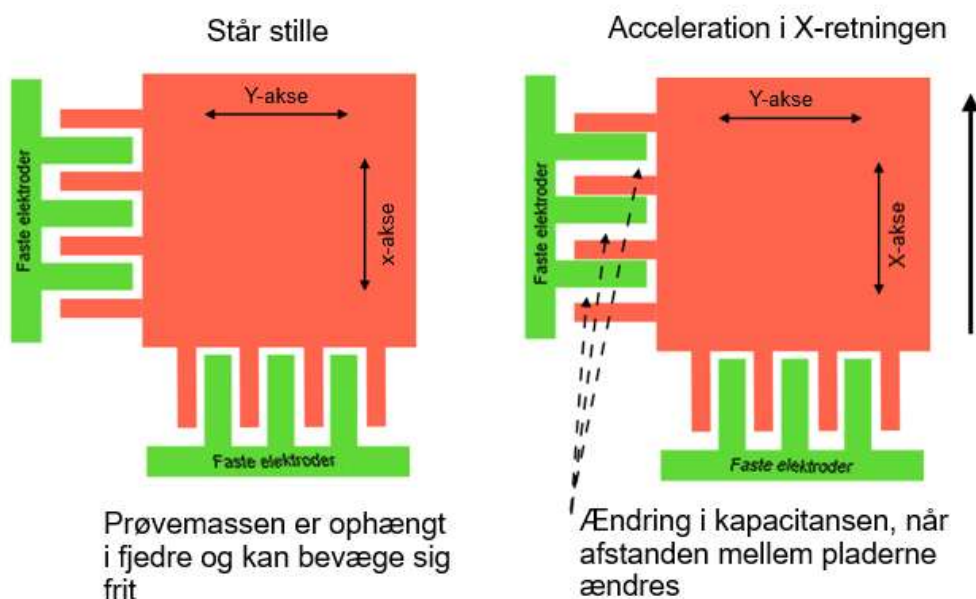
MPU-6050 og MPU-9250 er mikroelektromekaniske inertielle måleenheder (IMU), der kombinerer et tre-akset accelerometer og et tre-akset MEMS-gyroskop på samme kort, som vist i Figur 4. MPU-9250 inkluderer desuden et tre-akset magnetometer. De anvendes ofte i robotteknologi, droner og flight controllers, hvor præcis aflæsning af orientering og rotationsvinkler er kritisk. Dataoverførsel foregår via en I2C-bus, hvor sensoren fungerer som slaveenhed og sender digitale målinger til en mikrocontroller (ic-components, 2025).



Figur 4 MPU9250 / 6050

Accelerometeret i MPU-enhederne måler den kraft $\mathbf{F}$, som er summen af alle ikke-gravitationskræfter pr. masseenhed. Sensoren består af en kapacitiv MEMS-struktur, hvor en bevægelig masse (prøvemasse) er ophængt i fjedre og kan bevæge sig under acceleration. En fast elektrode række skaber en kapacitiv kobling til prøvemassen. Når enheden udsættes for acceleration, ændres kapacitansen, og denne ændring omdannes af en analog-til-digital-konverter (ADC) til en digital værdi. Figur 5 viser virkemåden af en kapacitiv MEMS-IMU. Disse værdier spænder fra 0 til 32.750 og udgør måleenhederne for accelerometeret.

Kapacitiv MEMS-sensor



Figur 5 Virkemåden af en kapacitiv MEMS-IMU

Gyroskopet fungerer analogt, men måler vinkelhastighed via Coriolis-effekten i stedet for lineær acceleration (White, 2019).

Ud fra accelerometerets data er det muligt at estimere hældningsvinkler relativt til jordens overflade. Matematisk kan hældningsvinklerne udledes vha. forholdet mellem accelerationskomponenterne.

$$\theta_{acc} = \arctan\left(\frac{a_y}{\sqrt{a_x^2 + a_z^2}}\right), \phi_{acc} = \arctan\left(\frac{-a_x}{a_z}\right) \quad (2.2.1)$$

Disse relationer er ikke-lineære og er meget følsomme over for støj, især under bevægelser.

Gyroskopet måler vinkelhastigheden ω omkring sensorens akser. Orienteringen estimeres ved integration af denne størrelse over tid:

$$\theta_k = \theta_{k-1} + \omega_k \cdot \Delta t \quad (2.2.1)$$

hvor Δt er måle tid og er følsom over for fejl og støj, hvilket kan resultere i betydelig drift over tid.

Problemet med orienteringsestimering fra IMU-data kan betragtes som et klassisk sensorproblem: accelerometeret giver et ok, men støjfyldt estimat, mens gyroskopet giver et relativt, men driftende estimat. For at kombinere disse data anvendes et komplementærfilter.

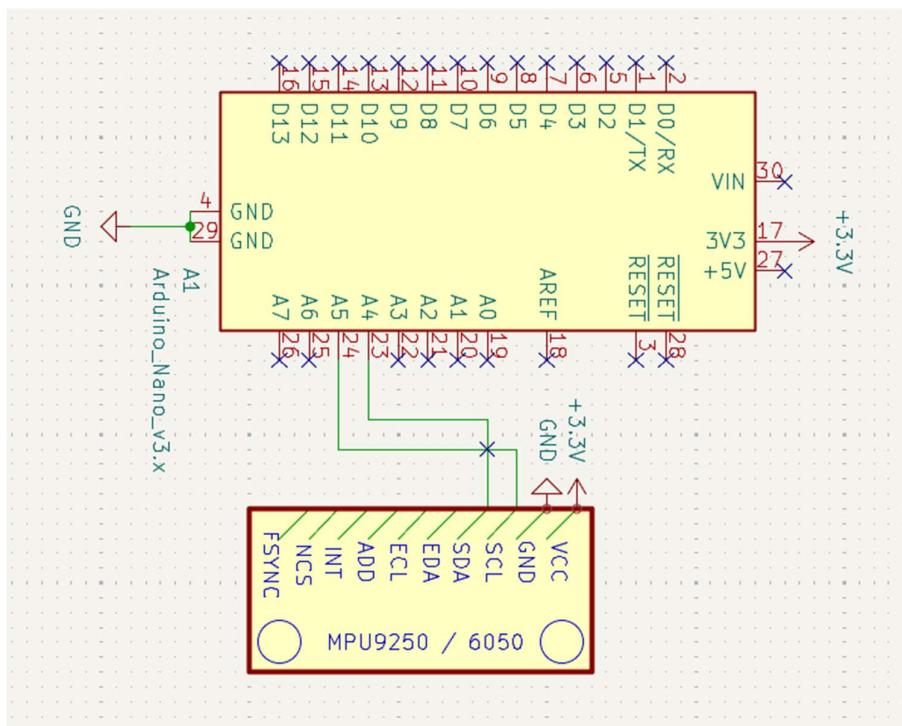
Filteret fungerer som en vægtet sum af de to målinger. Filteret udnytter, at gyroskopet er pålideligt ved høje frekvenser (hurtige ændringer), mens accelerometeret er pålideligt ved lave frekvenser (langsomme ændringer). Den diskrete form af komplementærfilteret kan beskrives ved:

$$\theta(t) = \alpha \cdot [\theta(t-1) + \omega \cdot \Delta t] + (1 - \alpha) \cdot \theta_{acc} \quad (2.2.2)$$

hvor α er en konstant mellem 0 og 1, typisk tæt på 1 (fx 0.92-98). Denne konstant bestemmer vægtningen mellem gyroskopet og accelerometeret. I praksis betyder dette, at gyroskopets signal dominerer på kort sigt, mens accelerometeret langsomt korrigerer drift (HiBit, 2023).

MPU-sensoren kræver en forsyningsspænding på typisk 3,3–5 V, hvilket forbindes til VCC-pinden, mens GND tilsluttes systemets jord.

Kommunikation med en mikrocontroller, eksempelvis en Arduino, foregår via I2C-bussen. Dette kræver, at sensorens SCL (Serial Clock Line) og SDA (Serial Data Line) forbindes til de tilsvarende I2C-pins på Arduinoen. Figur 6 viser sensoren forbundet til en mikrocontroller.



Figur 6 Skematisk tilslutning mellem MCU (Arduino Nano) og IMU (MPU9250 / 6050)

I2C-protokollen gør det muligt for mikrocontrolleren at fungere som master, der initialiserer læse- og skrivekommandoer, mens MPU-sensoren fungerer som slave. Når forbindelserne er etableret, skal sensoren initialiseres ved at skrive ønskede parametre til dens interne registre. Dette inkluderer bl.a. indstilling af gyroskop- og accelerometerfølsomhed via registrene 0x1B og 0x1C, samt aktivering af sensoren via power-management-registeret (typisk register 0x6B).

Når sensoren er initialiseret, begynder data læsning. Hver af de tre accelerometer- og gyroskopakser leverer målinger i to-byte registre, som kombineres til et 16-bit digitalt output. Disse data repræsenterer den målte acceleration i g eller vinkelhastighed i grader pr. sekund, afhængigt af den valgte følsomhed. Værdierne skal derefter normaliseres eller skaleres i forhold til den valgte følsomhed for at give præcise målinger. (InvenSense, 2012), (InvenSense, 2014).

Ved brug af en mikrocontroller som Arduino sker dette i et kontinuerligt loop, hvor data læses fra sensorens registre. I praksis betyder det, at Arduinoen gentagne gange sender en forespørgsel til sensorens I2C-adresse og modtager en række bytes, der repræsenterer de aktuelle målinger. Disse data kan derefter bruges direkte i algoritmer som komplementærfilteret for at beregne pitch, roll og yaw (HiBit, 2023).

2.3 PID-regulering

For at robotten kan balancere sig selv lodret op og beholde en samlet vinkel på $\theta = 0$, anvendes en PID-regulator. En PID-regulator er et automatisk styresystem, der minimerer forskellen mellem en ønsket værdi og en målt procesværdi, som eksempelvis afstand, temperatur eller tryk, ved hjælp af et feedback-loop.

Figur 7 viser det grundlæggende princip for PID-regulering.

2.3.1 Opbygning af PID-regulator

PID-regulatoren består af tre led: Proportional (P), Integral (I) og Derivat (D), der tilsammen sikrer stabil, hurtig og præcis styring uden fejl.

$$PID = P + I + D \quad (2.3.1)$$

P-leddet reagerer proportionalt med fejlen $e(t)$. Det betyder, at en større fejl giver et større bidrag til styresignalet (både positivt og negativt).

$$P = K_p \cdot e(t) \quad (2.3.2)$$

En ulempe ved P-regulering alene er, at systemet ikke kan eliminere fejlen fuldstændigt. Der skal altid være en vis fejl til stede for at skabe et styresignal, hvilket medfører en varig afvigelse som også kaldes for (steady-state fejl), hvor systemet stabiliserer sig tæt på, men ikke præcist på setpunktet.

I-leddet summerer fejlen over tid og sikrer dermed, at fejlen på sigt bliver nul.

$$I = k_I \cdot \int e(t) dt \quad (2.3.3)$$

Dette betyder, at selv små fejl over længere tid vil bidrage og blive fjernet. I-leddet fjerner derfor den steady-state fejl, som P-leddet efterlader. En ulempe er, at det kan gøre systemet langsommere og skabe oversving som ofte kaldes (overshoot), hvis det er for kraftigt.

D-leddet kigger fremad og forudsiger fejludviklingen, ved at differentiere fejlen.

$$D = K_D \cdot \frac{d}{dt} e \quad (2.3.4)$$

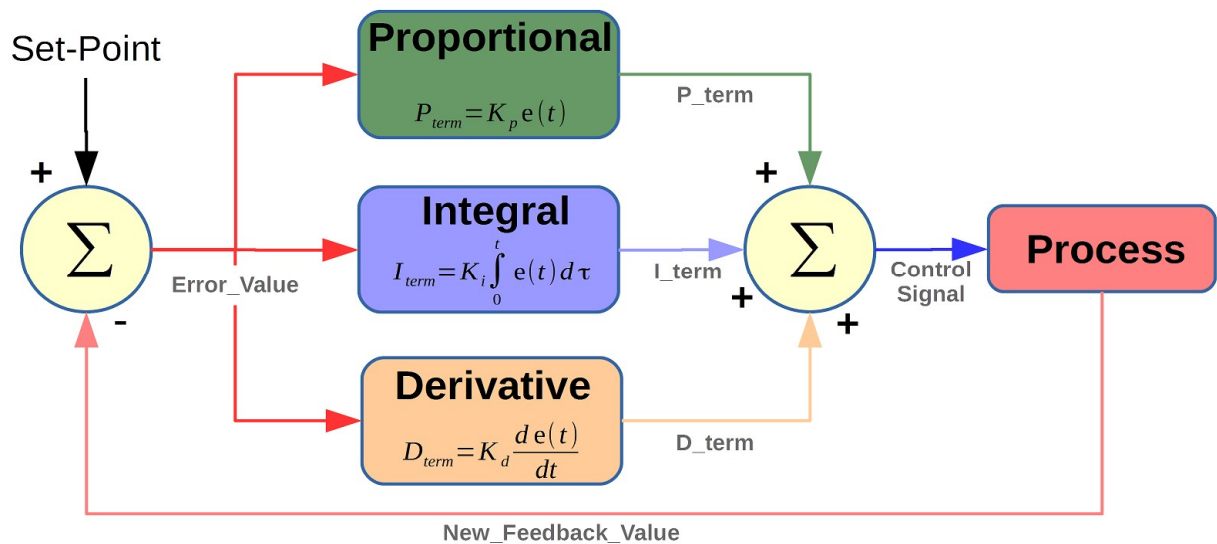
Det vil sige, at det reagerer på ændringshastigheden (hvor hurtigt fejlen vokser eller falder). Det modvirker for kraftige svingninger (overshoot) og dæmper reguleringen

Ved at summere alle tre led fås den samlede PID-regulator:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (2.3.5)$$

Hvor:

- $u(t)$ = styresignal (output)
- $e(t)$ = fejl
- K_p, K_i, K_d = konstanter (tuning-parametre)



Figur 7 Diagram af en PID-regulator (microcontrollerslab, 2019)

I praksis implementeres PID-regulatoren ofte i diskret form i en mikrocontroller, da systemet arbejder med tidsintervaller:

$$u = K_p \cdot e + K_i \cdot \sum e + K_d \cdot (e - e_{sidste}) \quad (2.3.6)$$

Her:

- $\sum e$ er summen af tidligere fejl (integral-leddet)
- $e - e_{sidste}$ er ændringen i fejlen (differential-leddet)

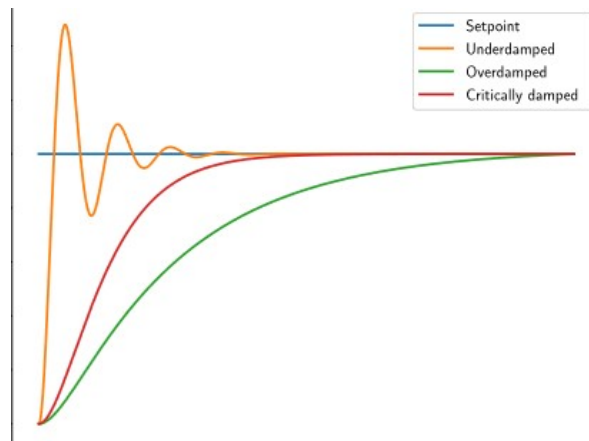
(microcontrollerslab, 2019)

2.3.2 Tuning af PID-regulatoren

For at en PID-regulator kan fungere optimalt, skal konstanterne K_p, K_i, K_d justeres korrekt. Denne proces kaldes tuning og har stor betydning for systemets stabilitet, præcision og hastighed. Formålet med tuning er at finde en balance, hvor systemet reagerer hurtigt på ændringer, men uden at skabe svingninger eller overshoot.

Konstanten K_p skal justeres nøjagtigt, idet en for høj K_p vil medføre unødvendige og høje oscillationer samt ustabilitet. Derimod vil en for lav K_p -værdi medføre et lag i systemet og gøre systemet langsomt og upræcist, ved ikke at reagere hurtigt nok på fejl. K_i skal tunes for at eliminere den fejl, som bliver introduceret af P-leddet i regulatoren, men hvis værdien er for høj, vil det medføre oversving og introducere ustabilitet. Til sidst skal K_d tilpasses for at dæmpe systemet ved at reagere hurtigt nok på ændringer i fejlen, men en for høj værdi kan gøre systemet følsomt over for støj og give urolige signaler. Figur 8 viser 3 forskellige PID-regulator:

- (orange-PID= K_p/K_i for høj; K_D for lav)
- (grøn-PID = K_p/K_i for lav; K_D for høj)
- (rød-PID - en ideal regulator med passende tunet parameter)



Figur 8 PID-regulatorer med forskellige tunet parametre

Tuningsprocessen kan udføres på flere forskellige måder. Den ene er eksperimentel, hvor man først begynder at øge K_p , indtil systemet begynder at oscillere, hvorefter den reduceres, indtil den er tilpasset korrekt. Derefter justeres K_i for at fjerne steady-state fejl, og til sidst finjusteres K_d for at opnå en stabil og dæmpet respons dette kaldes for Ziegler-Nichols-metoden. Alternativt kan man tune en PID-regulator ved hjælp af et

simulationsprogram såsom Matlab eller Simulink (microcontrollerslab, 2019).

Materialer

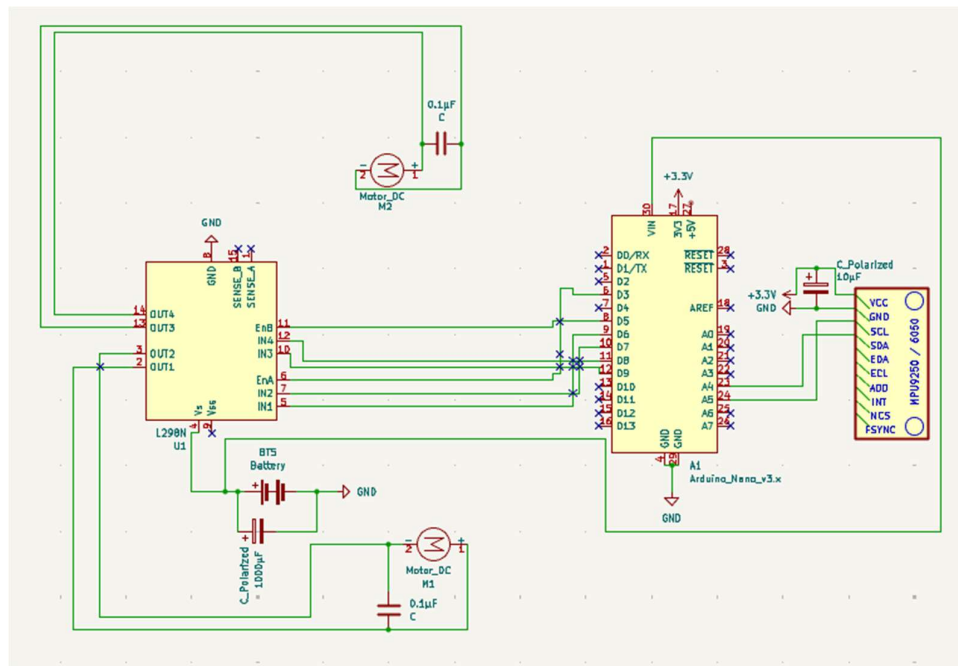
- Robot-komponenter
 - MPU9250 / 6050 sensor
 - Arduino Nano
 - 9V batteri.
 - Breadboard 400 huller
 - Jumper wires
 - Robot chassis
 - 2 x TT dc-motor 6-12V.
 - L298N Dual H Bridge Motor driver
 - 1 x RS PRO 1000 μ F Capacitor 35 V dc
 - 1 x RS PRO 100 μ F Capacitor 50 V dc
 - 2 x Capacitor Monolithic Ceramic 0.1 μ F 50V 104
 - Milwaukee Digital Vinkelmåler
- Software
 - Arduino IDE (script editor & compiler)
 - GeoGebra

Forsøgsopstilling

Her præsenteres den konkrete forsøgsopstilling, der danner grundlag for de målinger og resultater, der analyseres i de efterfølgende kapitler.

4.1 Robottens skematiske diagram

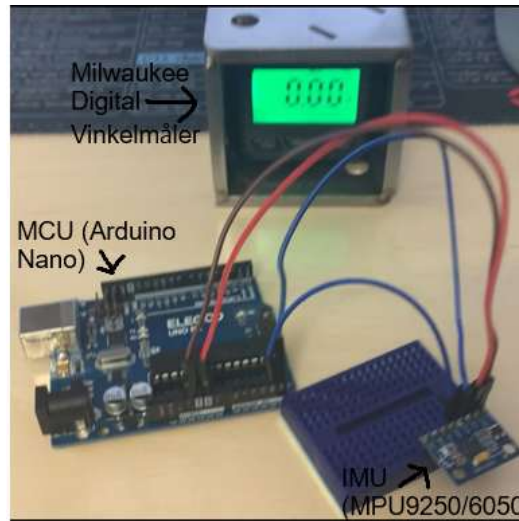
Det samlede skematiske diagram til robotten (figur 9)



Figur 9 Robottens skematiske diagram tegnet i KiCad.

4.2 Delforsøg #1

Fysisk forsøgsopstilling af delforøg 1 (karakterisering af MPU-9250/6050 sensoren og implementering af et komplementærfilter) -se figur 10



Figur 10 Fysisk forsøgsopstilling af delforøg 1

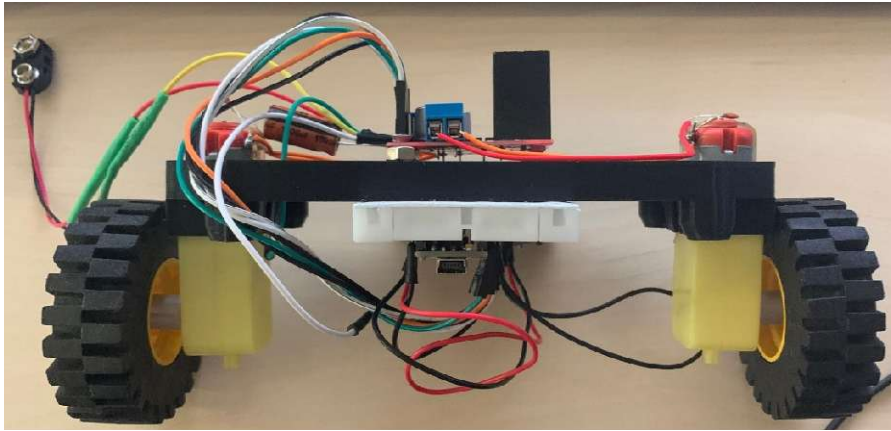
Delforøg 1 følger samme kredsløbsdiagrammet i figur 6.

4.3 Delforsøg #2 & #3

Fysisk forsøgsopstilling af delforøg 2 & 3 (Opstilling og justering af PID-parametre og analyse af systemets stabilitet) -se figur 11, figur 12 viser desuden robotten se fra siden:



Figur 11 Robottens opstilling i forsøg 2 og 3



Figur 12 Robottens opbygning ved forsøg 2 og 3

Fremgangsmåde

Følgende afsnit giver en detaljeret fremgangsmåde af hhv. Del forsøg 1, 2 og 3.

5.1 Delforsøg #1

- Trin 1. Tilslut MPU-sensoren til mikrocontrolleren (Arduino-Nano). VCC til 5 V. GND til GND. SDA til A4 og SCL til A5 - følger samme forbindelser som kredsløbet vist i figur 6. (Vigtig bemærkning: Nogle MPU-sensorer har ikke en indbygget spændingsregulator, og forventer max 3.3 V, giver man mere kan man risikere at brænde/ødelægge sensoren)
- Trin 2. Upload programmet som fremvises i bilag B til mikrocontrolleren som aflæser vinklen vha. accelerometer-data, vinklen vha. gyroskop-data, og vinklen ud fra en komplementærfilter.
- Trin 3. Placer IMU'en i kendte vinkler. For eksempel 0° - 20°
- Trin 4. Noter de målte vinkler givet fra sensoren samt den reelle vinkel målt med en kaliberet Milwaukee Digital Vinkelmåler, i GeoGebra.

5.2 Delforsøg #2

- Trin 1. Upload programmet som fremvises i bilag C som implementer en PID-regulator med en proportional, en integrerende og en differentierende del.
- Trin 2. Saml robotten som vist i hhv. Figur 11 og 12.

- Trin 3. Test robotten flere gange og juster PID konstanterne indtil robotten står stabilt ved den ønskede vinkel $\theta = 0$.
- Trin 4. Noter den målte hældnings vinkel af robotten givet fra sensoren som funktion af tid i GeoGebra.

5.3 Delforsøg #3

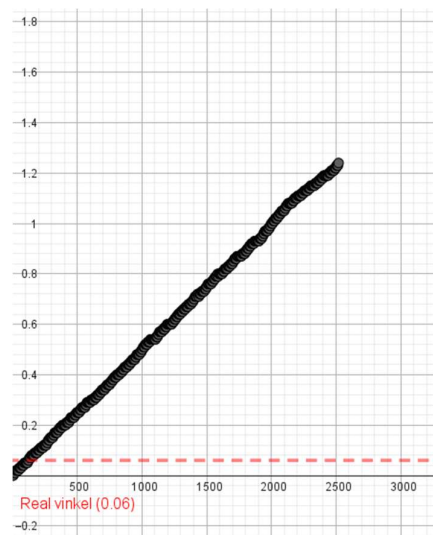
- Trin 1. Sørg for at robotten står stabilt med de PID-konstanter der blev fastlagt i delforsøg 2.
- Trin 2. Giv robotten et manuelt skub til en kendt startvinkel - fx 5° , 10° , 15° og 20° - ved at vippe den og slippe den.
- Trin 3. Aflæs via sensoren om robotten forsøger at genoprette balancen eller vælter, og noter den maksimale hældningsvinkel der blev registreret under genopretningen i GeoGebra.

KAPITEL 6

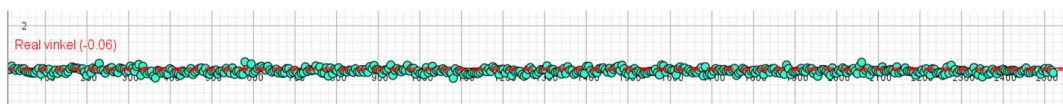
Resultater

6.1 Delforsøg 1

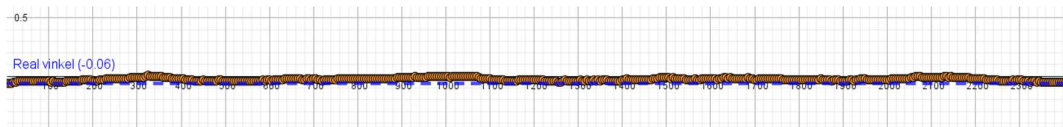
Til del forsøg 1 blev der foretaget 3 målinger, nemlig aflæsning af vinklen ud fra accelerometer-data (figur 14), gyroskop-data (figur 13), og til sidst vinklen ud fra en komplementærfilter hvor konstanten $\alpha = 0.98$. I alle tre målinger var vinklen ca. 0.06° aflæst fra en kaliberet Milwaukee Digital Vinkelmåler.



Figur 13 Vinklen ud fra gyroskop-data



Figur 14 Vinklen ud fra accelerometer-data

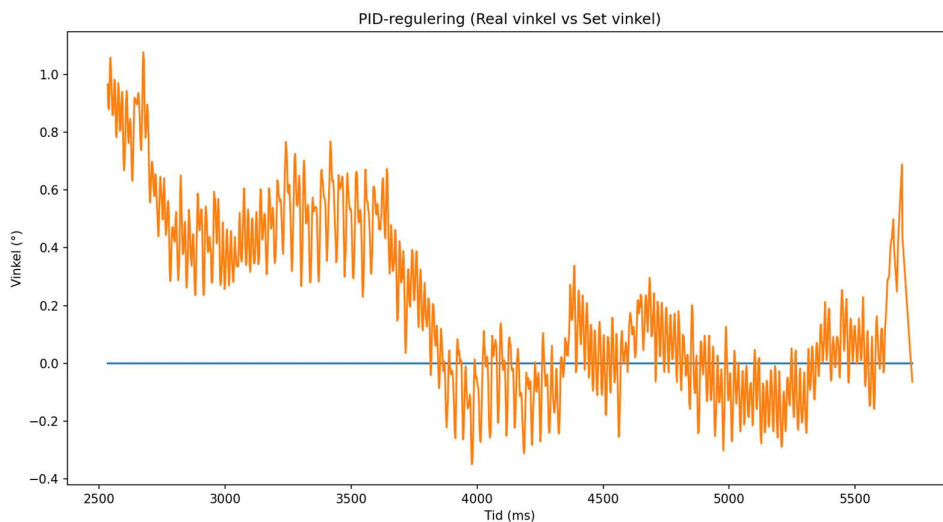


Figur 15 Vinklen ud fra en komplementærfilter hvor konstanten $\alpha = 0.98$.

6.2 Delforsøg 2

Til delforsøg 2 blev robottens hældningsvinkel målt, mens robotten forsøgte at stabilisere sig selv ved setpunktet. PID-regulatoren havde følgende tuning-parametre: $K_p = 15$; $K_i = 0.2$; $K_D = 10$

Figur 16 viser vinklen som funktion af tiden.

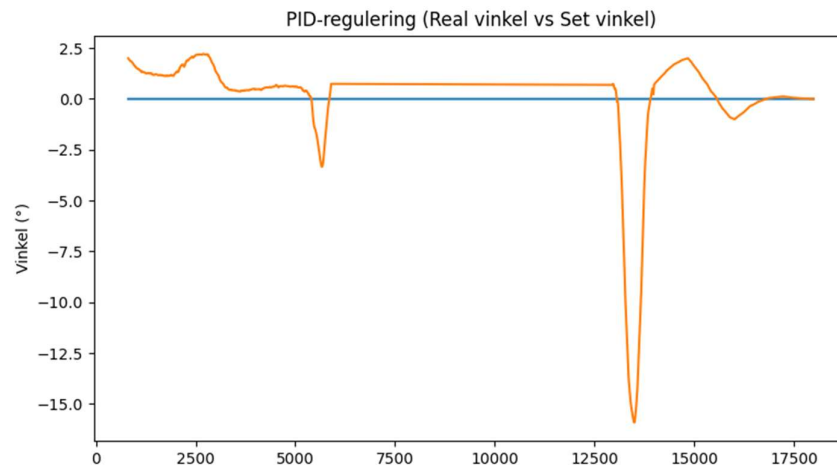


Figur 16 Robottens hældningsvinkel som funktion af tiden ved PID-regulering

6.3 Delforsøg 3

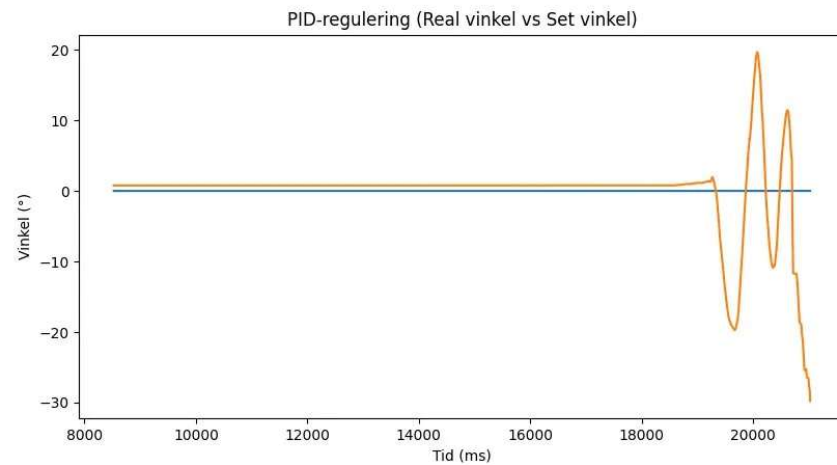
I delforsøg 3 blev robottens maksimale genopretningsvinkel θ_{max} , dvs. den største hældning hvorfra PID-regulatoren stadig er i stand til at stabilisere systemet bestemt.

Figur 17 viser vinklen som funktion af tiden, hvor robotten skubbes hhv. 2.5° , 3° og 15.5° og PID-regulatoren genopretter hurtigt balancen med dæmpede oscillationer der konvergerer mod 0° .



Figur 17 Vinkel som funktion af tiden, ved 3 forskellige forstyrrelser

Figur 18 viser vinklen som funktion af tiden, hvor robotten skubbes hhv. 20°, hvor robotten mister balancen og falder.



Figur 18 Vinkel som funktion af tiden, med en for stor forstyrrelse, der resulterer i at robotten falder.

Diskussion

Delforsøg 1 viser god overensstemmelse med den teoretiske forventning for alle tre målemetoder. Figur 13 viser hvordan vinklen beregnet udelukkende fra gyroskop-data drifter med tiden. Dette skyldes akkumulerede integrationsfejl, da gyroskopet integrerer vinkelhastigheden $\dot{\theta}$ over tid - selv en lille offset i målingen vil vokse lineært og gøre aflæsningen ubrugelig på længere sigt. Figur 14 viser den modsatte problematik for accelerometeret: vinklen svinger konstant omkring den reelle værdi som følge af mekanisk støj og vibrationer, hvilket bekræfter teorien om at accelerometeret er præcist på lang sigt men støjfyldt på kort sigt.

Figur 15 viser at komplementærfilteret kompenserer for begge svagheder ved at vægte gyroskopet højt ved hurtige bevægelser og lade accelerometeret korrigere for drift. Resultatet er en vinkelestimering der følger den reelle vinkel med små afvigelser, hvilket bekræfter at komplementærfilteret er den mest nøjagtige metode til vinkelestimering fra en IMU i dette system.

Delforsøg 2 viser i figur 16 at robotten stabilt holder sig oprejst ved en hældningsvinkel tæt på 0° , med maksimale udsving på ca. $\pm 2^\circ$. Dette indikerer velfungerende PID-tuning og korrekt implementering af reguleringssystemet. De små resterende udsving kan være pga. overfladefriktion samt begrænsning i motorernes PWM-signal.

Delforsøg 3 viser at robotten forsøger at genoprette balancen fra forstyrrelser på 2.5° , 3° og 15.5° , hvor vinklen i alle tilfælde konvergerer mod 0° , $\pm 2^\circ$ med dæmpede oscillationer som forventet fra teorien om et PID-reguleret inverteret pendul. Ved en forstyrrelse på 20° mister robotten derimod balancen og vælter, da PID-regulatoren ikke kan levere nok modkraft inden vinklen vokser eksponentielt i overensstemmelse med løsningen $\theta(t) = C_1 e^{\lambda t}$ i ligning 2.1.14. Den maksimale genopretningsvinkel kan derfor estimeres til:

$$15^\circ < \theta_{max} < 20^\circ$$

Dette resultat afspejler en fundamental begrænsning ved PID-regulering af et inverteret pendul: for store afvigelser fra setpunktet træder den ikke-lineære dynamik $\ddot{\theta} = \frac{mgl_c}{I} \sin(\theta)$ i kraft, og den lineariserede model som PID-regulatoren er designet ud fra, holder ikke længere.

En væsentlig kilde til usikkerhed i forsøget er målingen af hældningsvinklen. Den anvendte digitale vinkelmåler har en begrænset præcision, hvilket betyder, at den "reelle" referencevinkel ikke er helt korrekt. IMU-sensoren bidrager også med usikkerhed, idet accelerometeret er påvirket af vibrationer og støj, mens gyroskopet lider af drift over tid. Selvom komplementærfilteret reducerer disse fejl, vil der stadig være små afvigelser i den estimerede vinkel. Derudover afhænger PID-regulatorens ydeevne af måletiden Δt , som i praksis varierer en smule i Arduino-loopet, hvilket kan påvirke stabiliteten.

Derudover skal det bemærkes at systemets fysiske begrænsninger ligeledes har en påvirkning. De anvendte to DC-motorer har en begrænset omdrejningshastighed og drejningsmoment, hvilket betyder at regulatoren ikke altid kan levere den nødvendige kraft hurtigt nok. Batterispændingen falder desuden under høj belastning, hvilket reducerer motorernes ydelse. Et større batteri med højere strømkapacitet samt anvendelse af stepper-motorer, der tilbyder mere præcis og kraftfuld positionsstyring, ville kunne øge θ_{max} . Endelig ville en lavere placering af tyngdepunktet, eksempelvis ved at flytte batteriet ned mod hjulakslen, reducere vækstraten $\lambda = \sqrt{\frac{mg_c}{I}}$ og dermed gøre systemet lettere at stabilisere ved større hældninger.

Konklusion

Delforsøg 1 bekræftede at komplementærfilteret er den mest nøjagtige metode til vinklestimering fra IMU-sensoren, med små afvigelser fra den reelle vinkel. Delforsøg 2 viste at PID-regulatoren med de fundne tuning-parametre er i stand til at holde robotten stabilt oprejst ved 0° med udsving på maksimalt $\pm 2^\circ$. Delforsøg 3 fastslog at den maksimale genopretningsvinkel ligger i intervallet $15^\circ < \theta_{max} < 20^\circ$, og at systemet ved større forstyrrelser opfører sig i overensstemmelse med den teoretiske ustabile løsning.

Referencer

HiBit, 2023. *Hiblt*. [Online]

Available at: <https://www.hibit.dev/posts/92/complementary-filter-and-relative-orientation-with-mpu6050>

[Senest hentet eller vist den 5 April 2026].

ic-components, 2025. *ic-components*. [Online]

Available at: <https://www.ic-components.com/blog/mpu-6050-working-principle,application-and-package.jsp>

[Senest hentet eller vist den 4 April 2026].

InvenSense, 2012. *MPU-6050 Register Map and Descriptions Revision 4.0*, Sunnyvale: InvenSense Inc..

InvenSense, 2014. *MPU-9250 Product Specification*, San Jose: InvenSense Inc..

Johnson-Steigelman, H. T., u.d. *physicsthisweek*. [Online]

Available at: <https://www.physicsthisweek.com/math/small-angle-approximation/>

[Senest hentet eller vist den 10 April 2026].

microcontrollerslab, 2019. *microcontrollerslab.com*. [Online]

Available at: <https://microcontrollerslab.com/pid-controller-implementation-using-arduino/>

[Senest hentet eller vist den 1 April 2026].

Morin, D., 2007. *The Lagrangian Method*, Cambridge: Harvard Physics Department.

White, M., 2019. *mjwhite8119.github.io*. [Online]

Available at: <https://mjwhite8119.github.io/Robots/mpu6050>

[Senest hentet eller vist den 4 April 2026].

Bilag

Udledning af bevægelsesligning for den stive krop med et inertimoment I

Den kinetiske og potentielle energi for systemet er:

$$T = \frac{1}{2} I \dot{\theta}^2 \quad (\text{A.1})$$

$$V = mgl_c \cos(\theta) \quad (\text{A.2})$$

Den samlede Lagrangianen for systemet er:

$$L = \frac{1}{2} I \dot{\theta}^2 - mgl_c \cos \theta \quad (\text{A.3})$$

Lagrangianen indsættest nu i ligning 2.6.1 og der løses:

$$\begin{aligned} \frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \frac{\partial L}{\partial q_i} &= 0 \\ \frac{d}{dt} \left(\frac{\frac{1}{2} I \dot{\theta}^2 - mgl_c \cos \theta}{\partial \dot{\theta}} \right) - \frac{\frac{1}{2} I \dot{\theta}^2 - mgl_c \cos \theta}{\partial \theta} &= 0 \end{aligned} \quad (\text{A.4})$$

Der beregnes de nødvendige partielle afledte:

$$\frac{\partial L}{\partial \dot{\theta}} = \left(\frac{\frac{1}{2} I \dot{\theta}^2 - mgl_c \cos \theta}{\partial \dot{\theta}} \right) = I \dot{\theta} \quad (\text{A.5})$$

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}} \right) = \frac{d}{dt} \left(\frac{\frac{1}{2} I \dot{\theta}^2 - mgl_c \cos \theta}{\partial \dot{\theta}} \right) = I \ddot{\theta} \quad (\text{A.6})$$

$$\frac{\partial L}{\partial \theta} = \frac{\frac{1}{2} I \dot{\theta}^2 - mgl_c \cos \theta}{\partial \theta} = mgl_c \sin(\theta) \quad (\text{A.7})$$

Indsættes i Euler–Lagrange ligningen:

$$\underbrace{\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}} \right)}_{I \ddot{\theta}} - \underbrace{\frac{\partial L}{\partial \theta}}_{mgl_c \sin(\theta)} = 0 \quad (\text{A.8})$$

hvilket reduceres til:

$$\boxed{\ddot{\theta} = \frac{mg}{l} \sin(\theta)} \quad (A.9)$$

IMU kode til delforsøg 1

```
1. #include <Wire.h>
2.
3. #define IMU_ADDR 0x68
4. #define PWR_MGMT_1 0x6B
5. #define ACCEL_XOUT_H 0x3B
6.
7. float accelAngle = 0;
8. float gyroAngle = 0;
9. float compAngle = 0;
10.
11. unsigned long lastTime = 0;
12.
13. void imuReset() {
14.   Wire.beginTransmission(IMU_ADDR);
15.   Wire.write(PWR_MGMT_1);
16.   Wire.write(0x80);
17.   Wire.endTransmission();
18.   delay(150);
19.
20.   Wire.beginTransmission(IMU_ADDR);
21.   Wire.write(PWR_MGMT_1);
22.   Wire.write(0x00);
23.   Wire.endTransmission();
24.   delay(150);
25. }
26.
27. int readIMU(int16_t &ax, int16_t &ay, int16_t &az,
28.             int16_t &gx, int16_t &gy) {
29.   Wire.beginTransmission(IMU_ADDR);
30.   Wire.write(ACCEL_XOUT_H);
31.   if (Wire.endTransmission(false) != 0) return 0;
32.   if (Wire.requestFrom(IMU_ADDR, 14) != 14) return 0;
33.
34.   ax = (Wire.read() << 8) | Wire.read();
35.   ay = (Wire.read() << 8) | Wire.read();
36.   az = (Wire.read() << 8) | Wire.read();
37.   Wire.read(); Wire.read();
38.   gx = (Wire.read() << 8) | Wire.read();
39.   gy = (Wire.read() << 8) | Wire.read();
40.   return 1;
41. }
42.
43. void setup() {
44.   Serial.begin(115200);
45.   Wire.begin();
```

```

46.   imuReset();
47.   lastTime = millis();
48. }
49.
50. void loop() {
51.   int16_t ax, ay, az, gx, gy;
52.
53.   if (!readIMU(ax, ay, az, gx, gy)) return;
54.
55.   // Tid
56.   unsigned long now = millis();
57.   float dt = (now - lastTime) / 1000.0;
58.   lastTime = now;
59.
60.   // Konverter til fysiske værdier
61.   float ax_g = ax / 16384.0;
62.   float ay_g = ay / 16384.0;
63.   float az_g = az / 16384.0;
64.
65.   float gx_dps = gx / 131.0; // grader/sek
66.   float gy_dps = gy / 131.0;
67.
68.   // ----- ACCEL VINKEL -----
69.   accelAngle = atan2(ay_g, az_g) * 180.0 / PI;
70.
71.   // ----- GYRO VINKEL -----
72.   gyroAngle += gx_dps * dt;
73.
74.   // ----- KOMPLEMENTÆR FILTER -----
75.   float alpha = 0.98;
76.   compAngle = alpha * (compAngle + gx_dps * dt) + (1 - alpha) * accelAngle;
77.
78.   // Output
79.   Serial.print("Accel: ");
80.   Serial.print(accelAngle);
81.   Serial.print(" | Gyro: ");
82.   Serial.print(gyroAngle);
83.   Serial.print(" | Comp: ");
84.   Serial.println(compAngle);
85.
86.   delay(10);
87. }
88.

```

Balanceringskode til delforsøg 2 & 3

```
1. #include <Wire.h>
2.
3. // — IMU (MPU-6050) —————
4. #define IMU_ADDR    0x68
5. #define PWR_MGMT_1  0x6B
6. #define ACCEL_XOUT_H 0x3B
7.
8. // — IMU axis configuration —————
9. #define ACC_Y_SIGN   -1.0
10. #define ACC_Z_SIGN   -1.0
11. #define GYRO_Y_SIGN  -1.0
12.
13. // — Motor driver pins —————
14. #define ENA 3
15. #define IN1 6
16. #define IN2 7
17. #define ENB 5
18. #define IN3 8
19. #define IN4 9
20.
21. float PITCH_OFFSET = 0.0;
22. float MOTOR_TRIM   = 0.0;
23.
24. float Kp = 15.0;
25. float Ki = 0.0;
26. float Kd = 10.0;
27.
28. const float FALL_ANGLE = 30.0;
29. const float I_LIMIT    = 100.0;
30. const float DEAD_BAND  = 20.0;
31. const float CF_ALPHA   = 0.98;
32.
33. // — PID state —————
34. float pitch    = 0.0;
35. float integral = 0.0;
36. float prevError = 0.0;
37. unsigned long lastTime;
38.
39. // —————
40. // IMU helpers
41. // —————
42. void imuReset() {
43.   Wire.beginTransmission(IMU_ADDR);
44.   Wire.write(PWR_MGMT_1);
45.   Wire.write(0x80);
```

```

46. Wire.endTransmission();
47. delay(150);
48.
49. Wire.beginTransaction(IMU_ADDR);
50. Wire.write(PWR_MGMT_1);
51. Wire.write(0x00);
52. Wire.endTransmission();
53. delay(150);
54. }
55.
56. int readIMU(int16_t &ax, int16_t &ay, int16_t &az,
57.             int16_t &gx, int16_t &gy) {
58.     Wire.beginTransaction(IMU_ADDR);
59.     Wire.write(ACCEL_XOUT_H);
60.     if (Wire.endTransmission(false) != 0) return 0;
61.     if (Wire.requestFrom(IMU_ADDR, 14) != 14) return 0;
62.
63.     ax = (Wire.read() << 8) | Wire.read();
64.     ay = (Wire.read() << 8) | Wire.read();
65.     az = (Wire.read() << 8) | Wire.read();
66.     Wire.read(); Wire.read();
67.     gx = (Wire.read() << 8) | Wire.read();
68.     gy = (Wire.read() << 8) | Wire.read();
69.     return 1;
70. }
71.
72. // -----
73. // Motor helpers
74. // -----
75. void setMotors(float left, float right) {
76.     left = constrain(left, -255, 255);
77.     right = constrain(right, -255, 255);
78.
79.     digitalWrite(IN1, left >= 0 ? HIGH : LOW);
80.     digitalWrite(IN2, left >= 0 ? LOW : HIGH);
81.     analogWrite (ENA, abs((int)left));
82.
83.     digitalWrite(IN3, right >= 0 ? HIGH : LOW);
84.     digitalWrite(IN4, right >= 0 ? LOW : HIGH);
85.     analogWrite (ENB, abs((int)right));
86. }
87.
88. void stopMotors() {
89.     analogWrite(ENA, 0);
90.     analogWrite(ENB, 0);
91. }
92.
93. // -----
94. // Setup
95. // -----
96. void setup() {
97.     Serial.begin(115200);
98.
99.     pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
100.    pinMode(ENB, OUTPUT); pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
101.    stopMotors();
102.
103.    Wire.begin();

```

```

104. Wire.setClock(400000);
105. Wire.setWireTimeout(3000, true);
106.
107. imuReset();
108.
109. // Warmup
110. unsigned long warmup = millis();
111. while (millis() - warmup < 500) {
112.     int16_t ax, ay, az, gx, gy;
113.     if (readIMU(ax, ay, az, gx, gy)) {
114.
115.         float ayg = (ay / 16384.0) * ACC_Y_SIGN;
116.         float azg = (az / 16384.0) * ACC_Z_SIGN;
117.         float gys = (gy / 131.0) * GYRO_Y_SIGN;
118.
119.         float accPitch = atan2(ayg, azg) * 57.2958;
120.
121.         pitch = 0.98f * (pitch + gys * 0.005f)
122.             + 0.02f * accPitch;
123.     }
124.     delay(5);
125. }
126.
127. lastTime = millis();
128. Serial.println("Ready");
129. }
130.
131. // -----
132. // Main loop
133. // -----
134. void loop() {
135.     unsigned long now = millis();
136.     float dt = (now - lastTime) / 1000.0;
137.     lastTime = now;
138.
139.     if (dt < 0.002 || dt > 0.1) {
140.         stopMotors();
141.         return;
142.     }
143.
144.     int16_t ax, ay, az, gx, gy;
145.     if (!readIMU(ax, ay, az, gx, gy)) {
146.         stopMotors();
147.         return;
148.     }
149.
150.     // — Complementary filter with axis mapping —
151.     float ayg = (ay / 16384.0) * ACC_Y_SIGN;
152.     float azg = (az / 16384.0) * ACC_Z_SIGN;
153.     float gys = (gy / 131.0) * GYRO_Y_SIGN;
154.
155.     float accPitch = atan2(ayg, azg) * 57.2958;
156.
157.     pitch = CF_ALPHA * (pitch + gys * dt)
158.         + (1.0 - CF_ALPHA) * accPitch;
159.
160.     float error = pitch - PITCH_OFFSET;
161.

```



```

162. if (fabs(error) > FALL_ANGLE) {
163.     stopMotors();
164.     integral = 0.0;
165.     prevError = 0.0;
166.     return;
167. }
168.
169. integral = constrain(integral + error * dt, -I_LIMIT, I_LIMIT);
170. float derivative = (error - prevError) / dt;
171. prevError = error;
172.
173. float output = Kp * error + Ki * integral + Kd * derivative;
174. output = constrain(output, -255, 255);
175.
176. float drive = 0.0;
177. if (fabs(output) >= 1.0) {
178.     float sign = (output > 0) ? 1.0 : -1.0;
179.     drive = sign * (DEAD_BAND + (fabs(output) / 255.0) * (255.0 - DEAD_BAND));
180. }
181.
182. if (drive == 0.0) {
183.     stopMotors();
184. } else {
185.     float trimFactor = 1.0 + MOTOR_TRIM / 255.0;
186.     setMotors(drive, drive * trimFactor);
187. }
188.
189. Serial.print(now);
190. Serial.print(",");
191. Serial.print(0.0);    // setpoint
192. Serial.print(",");
193. Serial.println(error, 3);
194. }
195.

```